Day 2

90 Days SDET Interview Prep

Please make sure to take the printout of this particular document for studies.

90 Days SDET Interview Prep

Day 2

Contents

Q1: What is Constructor?

- A constructor is a special method in Java that is automatically invoked when an object is instantiated using the `new` keyword.
- It has the same name as the class and no return type not even void.
- The primary purpose of a constructor is to initialize the object's state by initializing the instance variables.

Example: `Person P = new Person("Jatin");`

```java
public class Person {

    // Instance variable
    private String name;

    // Constructor
    public Person(String name) {
        this.name = name; // Initializing Instance variable name
    }

    // Method to display data
    public void display() {
        System.out.println("Person name is: " + name);
    }

}
```

```java
public class Runner {

     public static void main(String[] args) {

       // Object creation using new keyword
       Person p = new Person("Jatin");   // Constructor is called here automatically

       p.display();
    }

}
```

Q2: What are different types of Constructors?

In Java we have 3 types of Constructors:

- **Default**
- **Parameterized**
- **Copy Constructor**

```java
public class Person {

    private String name;

    // Default Constructor
    public Person() {
        System.out.println("Default constructor called");
    }

    // Parameterized Constructor
    public Person(String name) {
        System.out.println("Parameterized constructor called");
        this.name = name;
    }

    // Copy Constructor which takes the other object reference as a parameter
    public Person(Person other) {
        System.out.println("Copy constructor called");
        this.name = other.name;
    }

    public void display() {
        System.out.println("Name: " + name);
    }

} // end of class
```

```java
public static void main(String[] args) {

        // Calls default constructor
        Person p1 = new Person();
        p1.display();

        // Calls parameterized constructor
        Person p2 = new Person("Jatin");
        p2.display();

        // Calls copy constructor
        Person p3 = new Person(p2);
        p3.display();
    }
```

- **Default Constructor (No-arg Constructor):**
  - This is automatically provided by Java if no constructor is explicitly defined.
    - Example `Person p = new Person();`
  - Once you define any constructor(Parameterized or Default) , Java no longer provides the default constructor.
- **Parameterized Constructor**:
  - A constructor that accepts parameters to initialize the values of the instance variable is called Parameterized Constructor
  - This provides flexibility in object creation as we can pass the value to instance variable during constructor call and is crucial for ***dependency injection.***

- **Copy Constructor:**
  - A constructor that takes the object of the same class as Parameter is called as Copy Constructor
    - Example :
      - `Person p1 = new Person("Raj");`
      - `Person p2= new Person(p1);` //p2 is clone of p1
  - This is useful for creating **deep copies** or cloning objects with specific modifications.

BONUS Question:

Deep Copy VS Shallow Copy

Copy Type	What Gets Copied	If You Change One Object

Shallow Copy	Only reference is copied.
	`Person p1 = new Person("Jatin");`
	`Person p2=p1`
		Both objects reflect the change

Deep Copy	A new object + new data is copied
	`Person p2= new Person(p1);`
	`Person p2= new Person(p1.name);`
		Objects become independent

Q3: Can constructor have a return type?

No, constructors cannot and should not have a return type not even void.

Remember:

If you add a return type to a constructor, Java treats it as a regular method that happens to have the same name as the class, but it won't be invoked during object instantiation.

Q4: Can constructor be static?

- No, constructors cannot be declared static in Java.
- Whenever you create an Object, Constructor Call is bound to happen!
- This would be a contradiction in terms ***because static members belong to the class rather than to any specific instance***, while constructors are specifically designed to create and initialize instances!
- Constructor belongs to the Instances!

Q5: Can constructor be private? When to use?

- Yes, constructors can and ***should be private in specific design patterns***.
- A private constructor prevents direct instantiation of the class from outside but within the class you can create the object!

Design Patterns where Constructors are private:

1. Singleton Pattern:

- When you want exactly one instance of a class in your application.
- Mostly used in `DatabaseManager` Class, `ConfigManager` where we just want a single instance of class for better memory management.

2. Utility Classes:

- Classes with only static methods (like Math, Collections, or custom utility classes) should have private constructors to prevent meaningless instantiation.
- If you plan to create any Utility Class which has all the methods as static make sure you make the constructor as private!

3. Builder Pattern (RequestSpec Builder and ResponseSpec Builder):

- If class has a lot of instance variables then object creation is gonna be bit complex because constructor will a lot of parameters.
- Builder pattern is used to **create complex objects step-by-step**, avoid constructor explosion, **make objects immutable (instance variables are final)**, and improve **readability and safety** of object creation.

4.Factory Pattern:

- When you want to control object creation through factory methods rather than direct instantiation.

```
if(browser.equals("chrome")) {
    driver = new ChromeDriver();
} else if(browser.equals("firefox")) {
    driver = new FirefoxDriver();
} else if(browser.equals("edge")) {
    driver = new EdgeDriver();
}
```

```
public class DriverFactory {

    private DriverFactory() {}  // prevent instantiation

    public static WebDriver getDriver(String browser) {

        switch (browser.toLowerCase()) {
            case "chrome":
                return new ChromeDriver();

            case "firefox":
                return new FirefoxDriver();

            case "edge":
                return new EdgeDriver();

            default:
                throw new IllegalArgumentException("Invalid browser: " + browser);
        }
    }
}
```

Q6: What is constructor chaining?

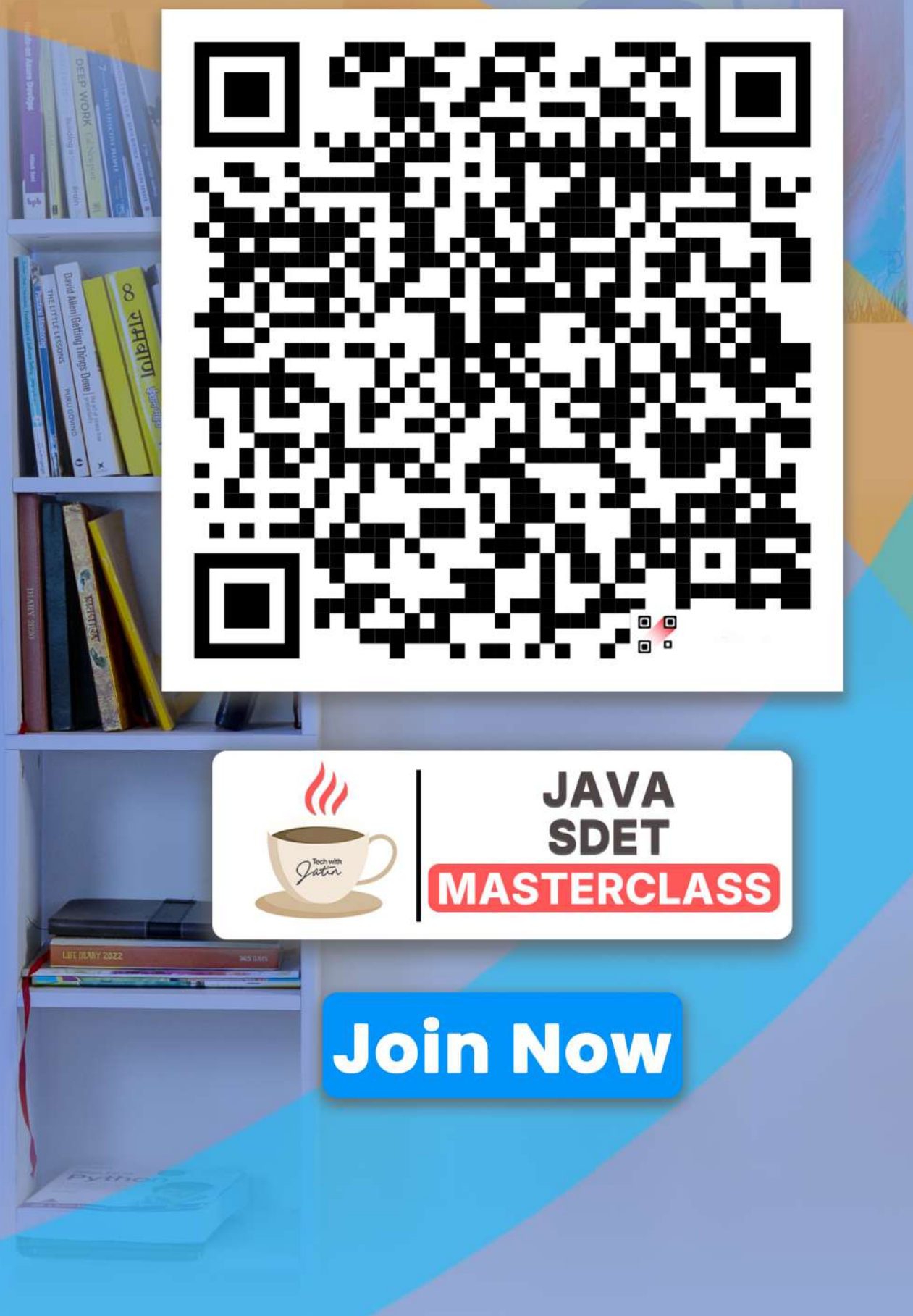
Constructor chaining is the practice of calling one constructor from another constructor within the same class or from the parent class.

Constructor Channing happens using **2 keywords**

Within the Class: this keyword

Parent and Child Classes: super keyword!

Tech with Jatin



JAVA
SDET
MASTERCLASS

Join Now